CMU 15-849 编译与压缩：从 CPU 代价模型到 RTX 5090 实测

算子融合、调度搜索、量化、剪枝与 LoRA 的论文复现

ShaneLiu04*

2026 年 10 月

摘要

本文是 CMU 15-849（Machine Learning Systems，论文研讨课）编译与压缩专题的复现报告。课程没有编程作业，本目录把 TVM/Ansor/TASO/XLA、PyTorch 2 (TorchInductor)/Triton、LLM.int8/SmoothQuant/GPTQ/AWQ、LoRA/QLoRA 与 Deep Compression 的核心算法做成可测试的最小实现：CPU 部分（反向模式自动微分、带融合 pass 的图 IR、带代价模型的调度搜索、量化/剪枝/LoRA 的数值实验，29 个 CPU 测试）给出模型；GPU 部分在 RTX 5090（sm_120）上给出测量。主要发现：(1) 访存受限链上 `torch.compile` 的加速与“eager 字节 / 融合字节”之比相差不到 7%（RMSNorm 9.86× vs 10.0×），融合后稳定在 1.41–1.61 TB/s；含 GEMM 的 MLP 只快 15%，实验室图 IR 的 roofline 预测落在实测的 4–6% 内；max-autotune 的 CUDA graph 输入拷贝让大张量访存受限 kernel 慢 2.08×。(2) 30 行 Triton matmul 在 8192^3 上达到 221.5 TFLOP/s（cuBLAS 214.3），但最优 tile 随形状变化；两个解析代价模型的 Spearman ρ 在 0.10–0.72 之间、各只抓住一种效应。(3) GPT-2 上 int8 per-channel 无损，int3 g128 时 GPTQ 把 ppl 从 65.92 拉回 37.24；SmoothQuant 的 α 选错（0.5）会让 per-tensor W8A8 崩到 407；int8 GEMM 是 bf16 的 3.0×（679 vs 227 TOP/s），batch-1 的 W8/W4 GEMV 快 1.8–2.0×/2.3–2.7×，只因读的字节少。(4) 一次性 50% 剪枝：幅值 538 ppl、Wanda 47；2:4 cuSPARSELt 在大 GEMM 上 1.54–1.58×，小 M 反而慢。(5) LoRA 把 GPT-2 medium 微调峰值显存从 16.07 降到约 12.1 GiB，差额正是 AdamW 状态；r=16 的 val loss 2.874 略好于全量 2.891。

目录

1 背景与作业要求 **3**

2 实验环境 **3**

3 方法与实现 **4**

3.1 代码结构 4
3.2 Q1：eager 字节计数器与融合基准 4
3.3 Q2：Triton matmul 网格与代价模型 4
3.4 Q3：GPT-2 上的 PTQ 5
3.5 Q4/Q5：剪枝与 LoRA 5

4 实验结果 **6**

4.1 Q0：CPU 侧基础——自动微分与图 pass 的正确性 6
4.2 Q1：算子融合——字节模型 vs 实测 6

*GitHub: `https://github.com/ShaneLiu04`。代码与原始指标：`https://gitee.com/liu-xingyan04/mlsys-infra`。本文基于公开课程材料独立完成，与课程团队无隶属关系。

4.3 Q2：调度/分块搜索——实测地形 vs 代价模型 10
4.4 Q3：量化——精度与速度 13
4.5 Q4：剪枝——精度与稀疏 kernel 速度 17
4.6 Q5：LoRA 与 QLoRA-like 微调 19
4.7 Q3 扩展：换到 7B——Qwen2.5-7B 上的同一套量化（RTX 6000D） 20

5 与理论模型的对照 **22**
5.1 融合：时间 = 字节 / 带宽 22
5.2 GEMM 分块：强度、共享内存与波次 22
5.3 weight-only 量化：batch 1 下时间 $\propto$ 权重字节 23
5.4 GPTQ 与 SmoothQuant 的公式 23
5.5 LoRA/QLoRA 的显存账 23

6 面试专题 **23**
6.1 20 秒结论 23
6.2 STAR 故事 23
6.3 追问 24

7 结论 **26**

1 背景与作业要求

15-849 是论文研讨课，没有官方编程作业。本实验把课程“编译器”与“压缩”两个单元的必读论文映射成五个可测量的问题，每个问题都同时有 CPU 侧的模型（已有实现）和 GPU 侧的测量（本次新增）：

问题	论文	本实验的 CPU 模型 → GPU 测量
Q1 算子融合	TVM [1]、XLA [4]、TASO [3]、PyTorch 2 / TorchInductor [5]	图 IR 的逐元素/epilogue 融合 pass 与 roofline [18] 字节模型 → eager vs `torch.compile`（default / max-autotune）、手写 Triton [6] 融合 kernel、编译时间与动态 shape 重编译
Q2 调度搜索	Halide [7]、TVM、Ansor [2]、Triton	单 SM 解析代价模型 + grid/random/进化搜索 → Triton matmul 的 192 点分块网格实测、autotune、cuBLAS、代价模型的 Spearman 排序相关
Q3 量化	LLM.int8 [8]、SmoothQuant [9]、GPTQ [10]、AWQ [12]	合成矩阵上的误差 → GPT-2 [19] 124M/355M 在 wikitext-2 [20] 上的困惑度；int8/FP8 GEMM 与 W8/W4 weight-only GEMV 吞吐
Q4 剪枝	Deep Compression [15]、NVIDIA 2:4 [16]、Wanda [17]	合成矩阵与小 MLP → GPT-2 困惑度、cuSPARSELt 2:4 与 CSR 的 kernel 速度
Q5 LoRA [13]	LoRA、QLoRA [14]	单层低秩拟合 → GPT-2 medium 全量微调 vs LoRA r=4..64 vs int8/int4 冻结基座，显存/步时/损失曲线/合并等价

表 1: 论文 → 问题 → 实现的对应。

每个数字都能追溯到 `artifacts/raw/*.csv`（CPU）或 `artifacts/raw/gpu/*.jsonl`（GPU）；派生量（加速比、Spearman 相关、实验室模型在 RTX 5090 参数下的预测）由 `scripts/plot_gpu.py` 写入 `artifacts/raw/gpu/summary_gpu.json`。

2 实验环境

CPU 侧	Windows Server，4 物理核 / 8 逻辑核；Python 3.12，numpy、torch CPU、pytest
GPU	NVIDIA GeForce RTX 5090（Blackwell，sm_120），170 SM，32 GB GDDR7，96 MB L2，驱动 595.71.05
GPU 峰值（规格）	bf16 稠密（fp32 累加）209.5 TFLOP/s；显存带宽 1.792 TB/s（实测 saxpy 约 1.55 TB/s）
共享内存	每 block 上限 101376 B（99 KB，Triton/Inductor 报错信息中给出）
软件	PyTorch 2.8.0+cu128，Triton 3.4.0，CUDA 12.8，transformers 5.21.0
服务器 CPU	仅用 2 个核（`taskset -c 47,48`，`OMP_NUM_THREADS=2`，Inductor 编译线程 2）
GPU 互斥	所有 GPU 命令经 `flock /root/autodl-tmp/gpu.lock`，计时不受其他实验干扰

表 2: 实验环境。RTX 5090 未收录在 `tools/mlsys_calc.py` 中，本实验在 `gpu/common.py` 中以同样的字段（bf16 峰值、带宽）定义它，并用 `mlsys_calc.GPU` 构造对象喂给实验室的代价模型。

计时方法：CUDA event 包住 n 次调用（n 自动选取使一组约 20–50 ms），取 5 组的中位数；预热 $n/5$ 次。编译相关实验设置 `TORCHINDUCTOR_FORCE_DISABLE_CACHES=1` 和全新的 `TRITON_CACHE_DIR`，使首调用时间是真实的冷编译成本。kernel 数用 `torch.profiler`（CUDA activity）数 5 次调用中的 kernel 事件（不含 memcpy/memset）取平均。

3 方法与实现

3.1 代码结构

```
compiler-compression/
  code/compkit/ autodiff, graph_ir (fusion/CSE/TASO merge), schedule_search,
      quantization, pruning, lora
  gpu/common.py CUDA-event timing, JSONL writer, RTX 5090 numbers
  gpu/triton_kernels.py fused RMSNorm, fused bias+GELU, tiled matmul (+autotune),
      W8A16/W4A16 GEMV
  gpu/fusion_bench.py Q1: chains / mlp / triton / dynamic
  gpu/matmul_search.py Q2: 192-config tile grid vs autotune vs cuBLAS
  gpu/llm_utils.py, quant_llm.py, quant_speed.py Q3: PTQ on GPT-2, low-precision
      GEMM/GEMV
  gpu/prune_bench.py Q4: one-shot pruning ppl, 2:4 cuSPARSELt and CSR speed
  gpu/lora_ft.py Q5: full FT vs LoRA vs QLoRA-like
  gpu/run_gpu.sh runner (each step under flock, taskset)
  scripts/plot_gpu.py all GPU figures + summary_gpu.json (local, no GPU)
  tests/test_gpu.py 11 tests: 7 CPU-runnable algorithm tests, 4 CUDA kernel tests (
      skip w/o CUDA)
```

3.2 Q1：eager 字节计数器与融合基准

关键问题是“eager 到底搬了多少字节”。我们用 `TorchDispatchMode` 拦截每个 aten 算子，把每个非视图算子的输入与输出张量字节数相加，这正是实验室图 IR `kernel_cost` 的同一模型（每个 kernel 读一次输入、写一次输出，无 L2 复用），只是作用在真实的 PyTorch 程序上：

```
class ByteCounter(TorchDispatchMode):
   def __torch_dispatch__(self, func, types, args=(), kwargs=None):
      out = func(*args, **(kwargs or {}))
      if func.overloadpacket.__name__ not in _VIEW_OPS:
         for t in unique_tensors(args, kwargs, out):
            self.bytes += t.numel() * t.element_size()
      return out
```

“融合理想字节”取图输入读一次加输出写一次（权重计入）。达到带宽 = 理想字节 / 实测时间；因此当理想字节小于 96 MB L2 时，同一输入被反复计时会命中 L2，“带宽”可以超过 GDDR7 峰值——这不是错误，而是图中黄色区域要说明的现象。

手写 Triton RMSNorm 每个 program 处理一行，整行一次装入寄存器，fp32 归约：

```
@triton.jit
def _rmsnorm_kernel(x_ptr, w_ptr, y_ptr, n_cols, eps, BLOCK: tl.constexpr):
   row = tl.program_id(0)
   cols = tl.arange(0, BLOCK); mask = cols < n_cols
   x = tl.load(x_ptr + row * n_cols + cols, mask=mask, other=0.0).to(tl.float32)
   w = tl.load(w_ptr + cols, mask=mask, other=0.0).to(tl.float32)
   rstd = tl.rsqrt(tl.sum(x * x, axis=0) / n_cols + eps)
   tl.store(y_ptr + row * n_cols + cols, (x * rstd * w).to(y_ptr.dtype.element_ty),
        mask=mask)
```

3.3 Q2：Triton matmul 网格与代价模型

`_matmul_kernel` 是标准的 grouped-ordering 分块 matmul（每个 program 一个 $B_M \times B_N$ 的 C 块，沿 K 以 B_K 步进，`tl.dot` 累加到 fp32）。网格为 $B_M, B_N \in \{32, 64, 128, 256\}$、$B_K \in \{32, 64\}$、

num_warps$\in\{4,8\}$、num_stages$\in\{2,3,4\}$，共 192 点；超出 99 KB 共享内存的配置被记录为 OOR（out of resource）。对比对象：同一 kernel 的 @triton.autotune（Triton 教程式的 10 个候选）与 cuBLAS（A @ B）。

两个代价模型对 (B_M, B_N, B_K) 三元组排序（实测取该三元组在 warps/stages 上的最好值），与实测时间算 Spearman ρ：

1. **实验室模型**：schedule_search.cost(Schedule("ijk", BM, BN, BK))，参数换成 RTX 5090 的每 SM 份额（209.5/170 TFLOP/s、1.792/170 TB/s、99 KB 缓存）。Triton 的一个 program 正是“ij 外层、k 内层”的循环序。
2. **波次量化 roofline**：$T = \lceil \#\text{tiles}/170 \rceil \cdot \max\left(\frac{2B_M B_N K}{P/170}, \frac{2(B_M+B_N)K}{BW/170}\right)$，只多考虑了“C 块在 170 个 SM 上按波次执行”。

3.4 Q3：GPT-2 上的 PTQ

所有 block 内线性层（每层 c_attn、attn.c_proj、c_fc、mlp.c_proj，GPT-2 124M 共 48 个）被量化，嵌入与 tied lm_head 保持 fp16。困惑度在完整 wikitext-2 test（287,644 token，按 1024 切成 280 个不重叠窗口）上计算；校准集为 train 中随机 128 个 1024-token 窗口。

GPTQ（自己实现）：按 block 顺序进行——第 l 个 block 的 Hessian $H = \frac{1}{n}\sum xx^\top$ 由已量化前 $l-1$ 个 block 的模型前向得到，然后对该 block 的四个线性层各自做列序 OBQ [11] 更新（lazy batch 128 列，阻尼 1% 平均对角）：

```
Hinv = torch.linalg.cholesky(torch.cholesky_inverse(torch.linalg.cholesky(H)),
    upper=True)
for i1 in range(0, cols, block):
   W1, Hi = W[:, i1:i2].clone(), Hinv[i1:i2, i1:i2]
   for i in range(i2 - i1):
      if (i1 + i) % g == 0: s, z = asym_params(W1[:, i:i + g], bits) # group grid
      q = asym_q(W1[:, i:i+1], s, z, bits)
      err = (W1[:, i:i+1] - q) / Hi[i, i]
      W1[:, i:] -= err @ Hi[i:i+1, i:] # push error onto unquantized columns
      Q1[:, i:i+1], E1[:, i:i+1] = q, err
   W[:, i2:] -= E1 @ Hinv[i1:i2, i2:] # lazy update of later blocks
```

AWQ 风格：输入通道缩放 $s_j = \overline{|x_j|}^a$，网格搜索 $a \in \{0, 0.1, \ldots, 1\}$，以代理误差 $\mathrm{tr}(\Delta W H \Delta W^\top)$ 选最优，复用 GPTQ 的同一 H。**W8A8（模拟）**：权重 per-channel int8；激活 per-tensor 或 per-token 动态 absmax int8；SmoothQuant 以校准集 $\max|X_j|$ 计算 $s_j = \max|X_j|^\alpha / \max|W_j|^{1-\alpha}$（$\alpha = 0.5$ 与 0.8）；LLM.int8 动态找出任一 token $|x| > 6$ 的输入列，这些列走 fp16，其余走 per-token×per-channel int8。所有 W8A8 都是“量化-反量化”模拟（fp16 GEMM），只评估精度。

速度方面：int8 用 torch._int_mm（int32 累加，与 fp64 参考逐元素比较），FP8 用 torch._scaled_mm（e4m3，张量 scale=1）；batch-1 的 weight-only 用自写 Triton GEMV：W8A16（每行 scale）与 W4A16（每字节两个 int4，group 128 的 fp16 scale，kernel 内解包反量化）。为避免 4096×4096 这类权重（32 MB）常驻 96 MB L2，“cold”模式轮换多份权重副本使工作集大于 384 MB。

3.5 Q4/Q5：剪枝与 LoRA

剪枝：幅值（每层全局阈值）30/50/70%、幅值 2:4 与 4:8、Wanda（$|W_{ij}| \cdot \|X_j\|_2$，按输出行比较）50% 与 2:4，均为一次性剪枝不微调；速度：to_sparse_semi_structured（cuSPARSELt 与 CUTLASS 两个后端）与 to_sparse_csr。LoRA：GPT-2 medium（355M），wikitext-2 train，batch 8×512，150 步，bf16 autocast；全量微调 AdamW lr 3×10^{-5}（fp32 主权重），LoRA 挂所有

block 线性层，$\alpha = 2r$，lr 10^{-3}，基座 bf16 冻结；QLoRA-like 把冻结基座存为 int8（per-channel）或 int4（g128 打包），用自定义 `autograd.Function` 在前向与反向中各反量化一次，只保存量化张量，否则 `F.linear` 会把反量化出的 bf16 权重存进计算图、显存毫无节省。

4 实验结果

4.1 Q0：CPU 侧基础——自动微分与图 pass 的正确性

两层 MLP + softmax 交叉熵上，自写反向模式自动微分的梯度与 torch 最大差 $\leq 5.6 \times 10^{-17}$，与中心差分最大差 $\leq 2.3 \times 10^{-10}$（`autodiff_check.csv`）；拓扑序是迭代实现，5000 层链式乘法反向不爆栈（测试）。图 IR 的常量折叠、CSE、逐元素融合、epilogue 融合前后解释器输出最大差为 0，TASO 式 matmul 合并后为 1.7×10^{-18}（`graph_pass_checks.csv`）。这些是后续“字节模型”可信的前提：pass 不改变语义，只改变 kernel 边界。

4.2 Q1：算子融合——字节模型 vs 实测

CPU 模型（H100 参数）。 图 1 是实验室图 IR 在 T=4096, d=4096, d_{ff}=16384 的 MLP 上的预测：不融合 7 个 kernel、1.376 GB；逐元素融合 5 个、1.040 GB（-24%）；再加 GEMM epilogue 3 个、0.705 GB（-49%），但 roofline 时间只从 1342 µs 降到 1132 µs（-16%），因为两个 GEMM（各 556 µs）是计算受限的。这给出了一个可检验的预测：*融合对访存受限链收益巨大，对含大 GEMM 的块收益很小。*

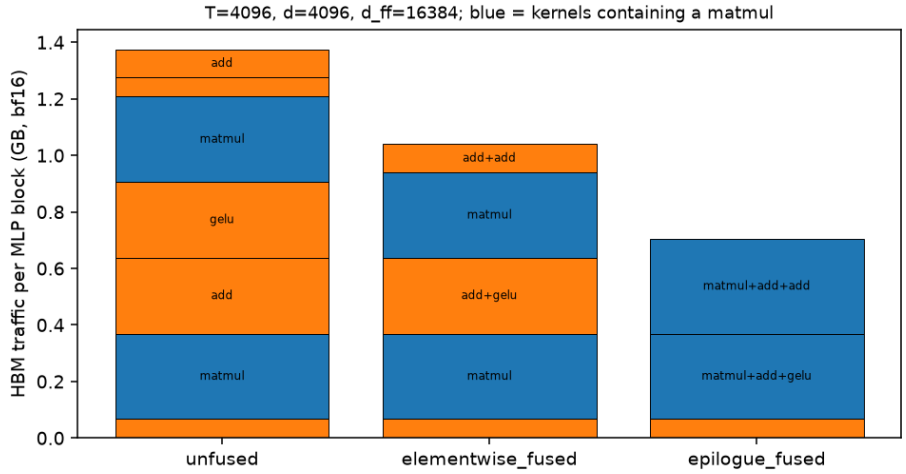


图 1: （CPU 模型）实验室图 IR 的融合 pass 对 Transformer MLP 的 HBM 流量与 roofline 时间的预测。

GPU 实测：访存受限链。 图 2 to 4 是五条链在 $M \in \{256, 1024, 4096, 16384\}$、$N$=4096、bf16 上的结果。最大尺寸（每个张量 128 MB，超出 L2）下：

链	eager kernel	eager µs	Inductor µs	实测加速	字节比（eager/理想）	Inductor TB/s
RMSNorm（fp32 手写式）	9	1723.8	174.9	9.86×	10.0×	1.54
softmax（fp32 上转）	3	885.4	175.3	5.05×	5.0×	1.53
bias+GELU+dropout	3	580.3	190.1	3.05×	3.25×	1.41
残差链（4 输入）	5	1116.7	417.2	2.68×	2.60×	1.61
LayerNorm（`F.layer_norm`）	1	176.0	175.0	1.01×	1.00×	1.53

表 3: M=16384 时 eager 与 Inductor（default 模式）的对比。字节比由 dispatch 计数器（eager）与“输入读一次 + 输出写一次”（理想）算出，`fusion_chains.jsonl`。

结论：字节比就是加速比

在不命中 L2 的尺寸上，Inductor 相对 eager 的实测加速与“eager 字节 / 理想字节”的比值相差不到 7%（RMSNorm 9.86× vs 10.0×，softmax 5.05× vs 5.0×，残差链 2.68× vs 2.60×）。融合后所有链都稳定在 1.41–1.61 TB/s，即 GDDR7 峰值的 79–90%、saxpy 实测的 91–104%——融合 kernel 已经顶在带宽墙上。这正是实验室 `kernel_cost` 的模型：访存受限算子的时间 = 字节 / 带宽，融合只是删掉中间张量的写 + 读。`F.layer_norm` 在 eager 里本来就是一个手写融合 kernel，所以 Inductor 没有可省的字节（1.01×）。

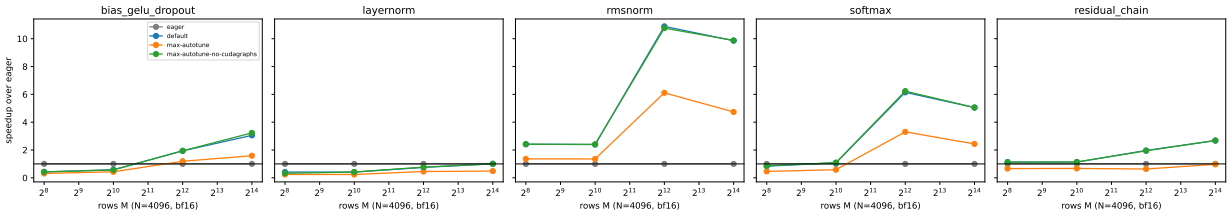


图 2: 各链相对 eager 的加速比随行数 M 的变化（N=4096）。小尺寸下编译版本更慢：每次调用约 23–27 µs 的 Python/guard 开销（eager 单 kernel 约 11 µs），max-autotune 的 CUDA graph 还要先把输入拷进静态缓冲区。

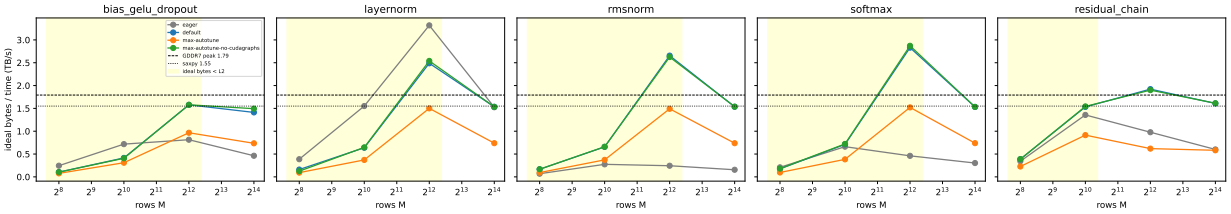


图 3: 达到带宽 = 理想字节 / 时间。黄色区域：理想字节小于 96 MB L2，反复计时同一输入会命中 L2，所以 LayerNorm M=4096（67 MB）eager 达到 3.32 TB/s、RMSNorm/softmax Inductor 达到 2.66/2.83 TB/s，高于 GDDR7 峰值。M=16384 时所有融合版本回落到 1.53 TB/s 左右。

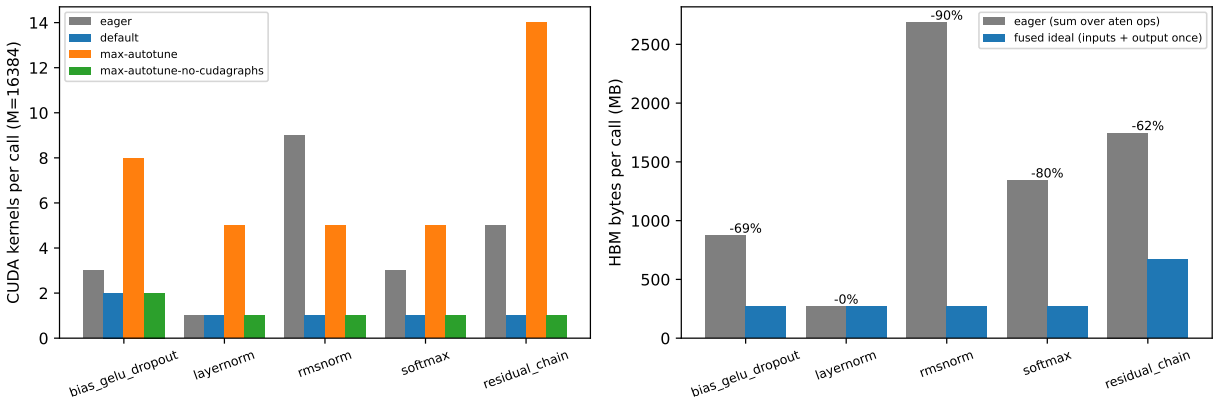


图 4: 左：M=16384 时每次调用的 CUDA kernel 数。default 与 no-cudagraphs 都把每条链压成 1–2 个 kernel；max-autotune（开 CUDA graph）多出 3–13 个 kernel，是 cudagraph tree 把非静态输入拷入图内缓冲区的 copy kernel。右：eager 字节与融合理想字节，标注省下的比例（RMSNorm 90%、softmax 80%、bias+GELU+dropout 69%、残差链 62%、LayerNorm 0%）。

一个反直觉的结果：max-autotune 让访存受限链变慢 2×。 M=16384 时 max-autotune 的 RMSNorm/softmax/LayerNorm 都是约 364 µs，default 是约 175 µs；而 `max-autotune-no-cudagraphs` 与 default 相同（174.5 µs）。区别只在 CUDA graph：输入张量不是图的静态地址，每次 replay 前要先拷贝，对一个 128 MB 输入、128 MB 输出的 kernel 来说，这次拷贝正好把流量翻倍。残差链有 4 个输入，max-autotune 达到 1152.7 µs，比 eager（1116.7 µs）还慢。CUDA graph 省的是 launch 开销（每 kernel 几 µs），只在 kernel 很短、调用很密（decode）时值得；对大张量单 kernel 它是负优化。

含 GEMM 的 MLP 与 Transformer block。 图 5 与表 4。d=1024、d_{ff}=4096，结构与实验室 `transformer_mlp` 完全相同（LN → W1+b1 → GELU → W2+b2 → 残差）。

（T=4096）	kernel 数	时间 µs	TFLOP/s	首调用/编译 s	与 eager 最大差
eager	5.0	419.4	163.9	–	0
Inductor default	5.0	414.4	165.8	0.59	0.031
Inductor max-autotune	4.0	366.8	187.4	6.91	0.031
max-autotune-no-cudagraphs	2.8	356.8	192.6	6.45	0.031
实验室模型（5090 参数）不融合	7	435.7	–	–	–
实验室模型逐元素融合	5	388.9	–	–	–
实验室模型 + epilogue	3	338.1	–	–	–

表 4: Transformer MLP，T=4096（`fusion_mlp.jsonl`；模型行来自 `summary_gpu.json` 中 `lab_mlp_model_5090`）。

结论：实验室模型对 MLP 的预测落在实测的 4% 内

实验室图 IR 在 RTX 5090 参数（209.5 TFLOP/s、1.792 TB/s）下预测不融合的 MLP 为 435.7 µs，eager 实测 419.4 µs（低 3.7%）；预测全融合为 338.1 µs，max-autotune-no-cudagraphs 实测 356.8 µs（高 5.5%）。eager 只有 5 个 kernel 而不是模型的 7 个，因为 cuBLASLt 的 `addmm` 本来就把 bias 融进了 GEMM epilogue。default 模式几乎没有收益（414.4 vs 419.4 µs，−1.2%）：它只把 GELU 等逐元素算子挪了位置，GEMM 仍调 cuBLAS；max-autotune 的 −15% 主要来自 Triton GEMM 模板把 GELU/残差做进 epilogue 并选到更好的 tile，代价是编译时间从 0.59 s 涨到 6.91 s（11.7×）。

Block（含 SDPA）同样：eager 712.4 µs，default 694.3 µs，max-autotune 621.6 µs，no-cudagraphs 607.0 µs（−14.8%），编译 12.2–12.9 s。

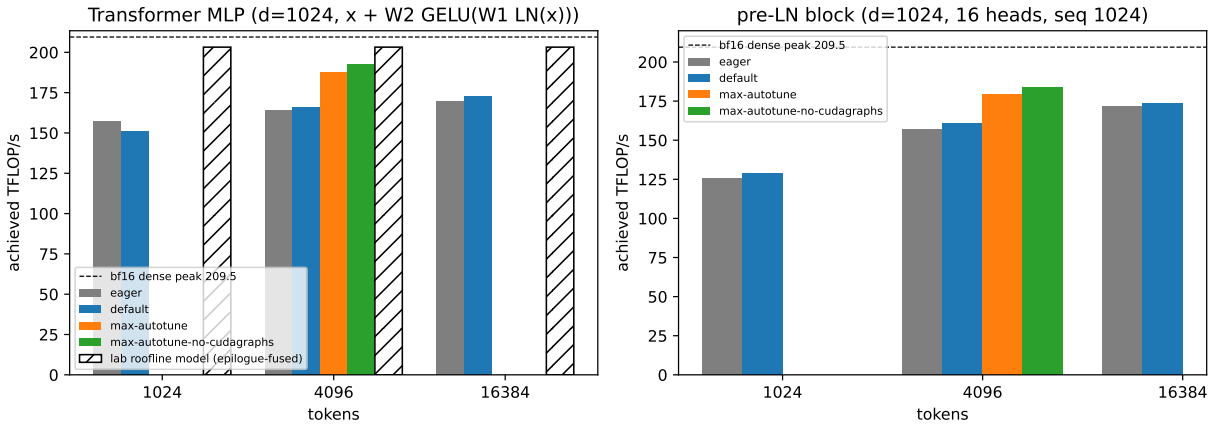


图 5: MLP 与 block 的达到 TFLOP/s。MLP 图中斜线柱是实验室 roofline 模型（epilogue 融合）的预测吞吐。GEMM 占主导时所有版本都在 126–193 TFLOP/s（峰值的 60–92%；最低的是 1024 token 的 block，attention 与小 GEMM 占比大），融合只能撬动剩下的 10–15%。

手写 Triton vs Inductor。 图 6。大尺寸（16384 × 4096）上手写 RMSNorm 162.3 µs、Inductor 175.0 µs、eager 1725.3 µs；bias+GELU 三者里 Triton 与 Inductor 相同（176.1 vs 175.6 µs），都是 1.52–1.53 TB/s。结论：对逐元素/行归约这一类，Inductor 生成的 kernel 与手写 kernel 只差 0–7%，手写的价值在于控制（如整行驻留寄存器、跳过 Inductor 的 guard），而不是更高的带宽上限。小尺寸下手写 kernel 的 launch 下限约 16–20 µs（Triton Python launcher），Inductor 约 23–25 µs，eager 单 kernel 约 11.8 µs。

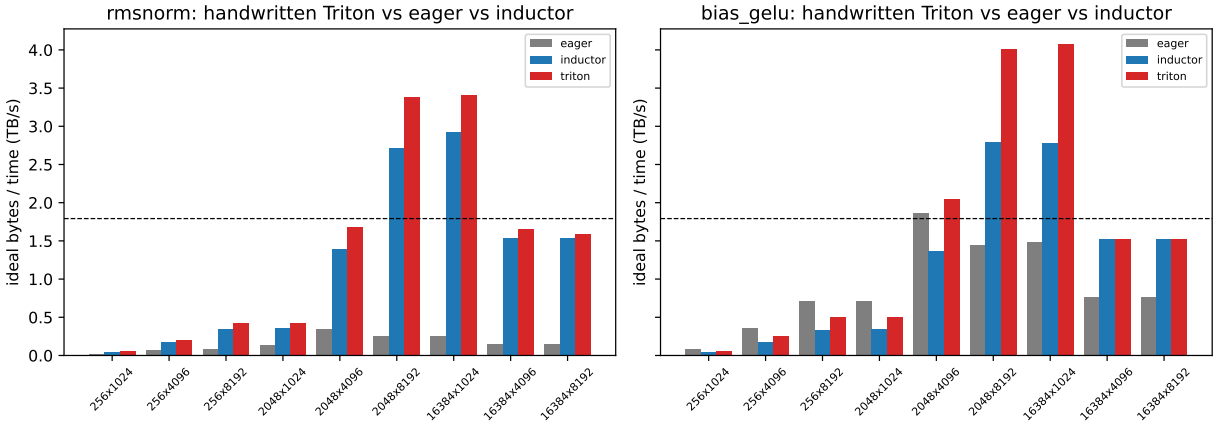


图 6: 手写 Triton 融合 kernel 与 eager、Inductor 的达到带宽（`fusion_triton.jsonl`）。16384×1024 与 2048 × 8192（32 MB，L2 内）时手写 bias+GELU 达到 4.01–4.07 TB/s，即 L2 带宽量级。

编译成本与动态 shape。 图 7。对一个 MLP 依次喂 10 个不同的序列长度（512, 768, 1024, 1536, 2048, 3072, 4096, 640, 1024, 2048）：

策略	编译出的图	首调用总耗时	说明
`dynamic=False`	8	4.46 s	每个新长度重编译，到 `cache_size_limit`=8 为止
默认（automatic dynamic）	2	0.99 s	第 1 次静态，第 2 次发现长度变化后编译一个符号形状图
`mark_dynamic(x, 0)`	1	0.54 s	一开始就编符号形状图

表 5: 动态 shape 的重编译代价（`fusion_dynamic.jsonl`）。

稳态时间三者相同：4096 token 时 static 419.8 µs、automatic 420.4 µs、mark_dynamic 420.2 µs，eager 425.7 µs。即对 GEMM 主导的模型，符号形状图没有可见的性能损失，而静态策略把 3.9 s 花在了 7 次重编译上。工程上的结论：serving 中长度变化的维度应 `mark_dynamic`，或者把长度 pad 到少量 bucket。

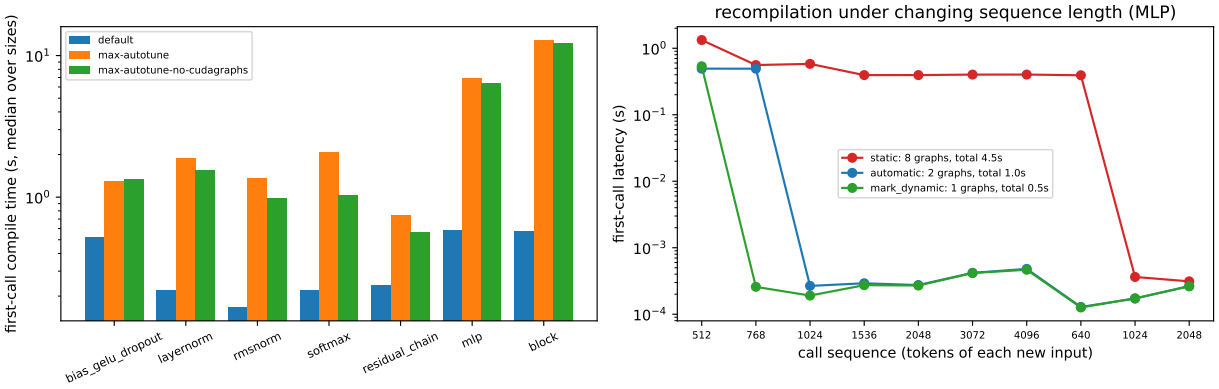


图 7: 左：各模式的首调用编译时间（各尺寸中位数，对数轴）。逐元素链 default 0.17–0.52 s，max-autotune 0.75–2.1 s；MLP/block 的 max-autotune 为 6.9/12.9 s（要 benchmark 约 20 个 GEMM 模板，其中不少因需要 128–144 KB 共享内存而在 sm_120 的 99 KB 上直接失败）。右：动态 shape 序列中每次首调用的延迟。

4.3 Q2：调度/分块搜索——实测地形 vs 代价模型

CPU 模型。 图 8：实验室的 2048^3 空间（6 种循环序 $\times$ 8^3 种 tile = 3072 个调度）在单 H100 SM 代价模型下只有 8 个落在最优 5% 内；达到 5% 以内所需的评估次数：按枚举顺序的 grid 434 次，random 中位 244 次（P90 649），进化搜索中位 147.5 次（P90 496），进化约快 1.65×。最优 ijk/256×256×64 为 5.99 ms（计算下限 2.29 ms），因为 C 块强度 $t_m t_n/(t_m + t_n) = 128$ FLOP/B 低于该模型的脊点 295。

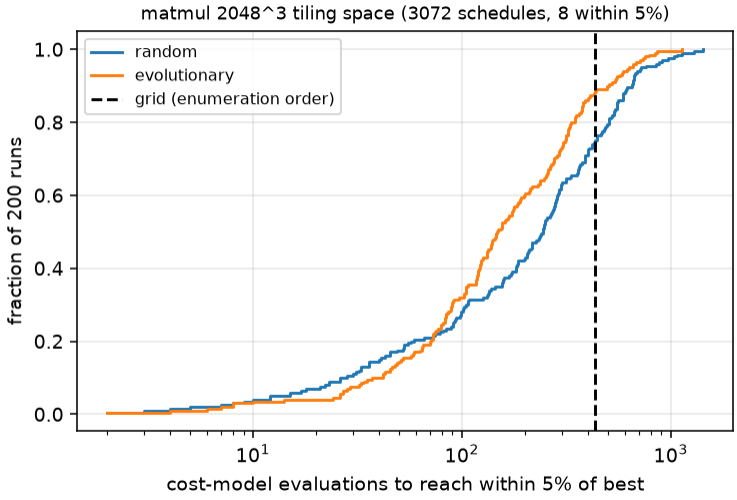


图 8:（CPU 模型）200 次重复中达到最优 5% 以内所需评估次数的 CDF（`schedule_runs.csv`）。

GPU 实测地形。 图 9 and 10 与表 6。

形状（fp16）	cuBLAS	网格最优（配置）	autotune（选中配置）	5% 内配置数 / 可运行数
4096^3	209.6	211.8（64×64×64, w4, s3）	206.7（64×128×32, w4, s4）	29 / 176
8192^3	214.3	221.5（128×128×32, w4, s3）	214.7（128×256×64, w8, s3）	17 / 96
$256 \times 4096 \times 4096$	175.0	165.0（128×64×64, w8, s4）	152.3（128×64×32, w4, s4）	8 / 176

表 6: Triton matmul 网格（TFLOP/s，`matmul_search.jsonl`）。8192^3 跳过了 B_M 或 B_N=32 的配置（不可能有竞争力且每个要数秒）。OOR 配置在 4096^3 上 16 个，全部是 B_K=64 且 stages 3–4 的大 tile，所需共享内存 110–196 KB > 99 KB。

结论：一个 30 行的 Triton kernel 可以追平 cuBLAS，但最优 tile 随形状变化

方阵上网格最优比 cuBLAS 快 1.0%（4096^3）与 3.4%（8192^3），221.5 TFLOP/s 甚至高于规格表的 209.5 TFLOP/s——实卡运行在高于标称 boost 的频率上，所以“峰值”应以实测为准。但最优 tile 在三个形状上各不相同：4096^3 偏好 64×64（$64 \cdot 64 = 4096$ 个 C 块，约 24.1 波，尾波浪费小），8192^3 偏好 128×128，瘦长的 M=256 偏好 128×64。autotune 只在 10 个教程候选里挑，因此方阵上落后网格最优 2.4–3.1%，瘦长形状上落后 7.7%，落后 cuBLAS 13%。

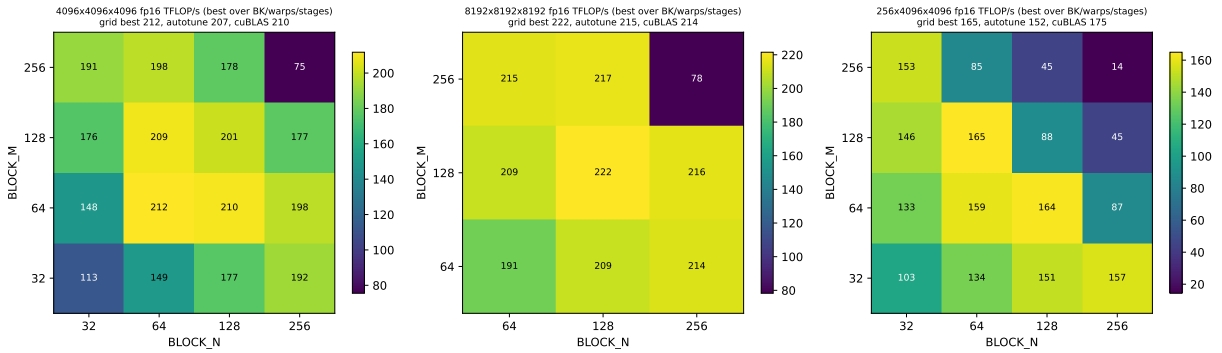


图 9: $B_M \times B_N$ 热力图（对 B_K/warps/stages 取最好值）。256×256 一列在三个形状上都是最差（75/78/14 TFLOP/s）：每个 program 的累加器 256 × 256 fp32 = 256 KB 寄存器需求，4–8 个 warp 装不下，发生寄存器溢出；且只有 256 个（4096^3）甚至 16 个（M=256）C 块，170 个 SM 用不满。

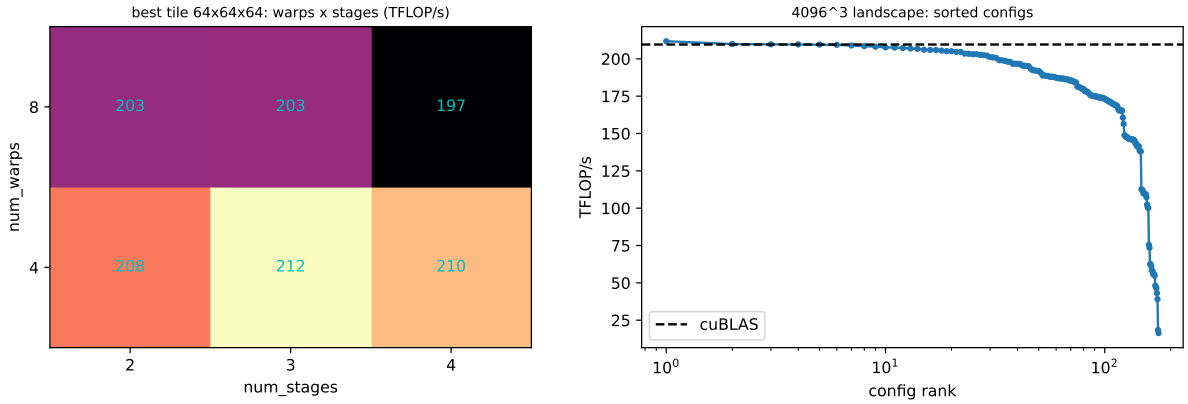


图 10: 左：4096^3 最优 tile（64×64×64）上 warps×stages 的影响。右：176 个可运行配置按吞吐排序，中位数 175.2 TFLOP/s，只有 29 个在最优 5% 内。

代价模型能否排对？ 图 11。对 (B_M, B_N, B_K) 三元组求 Spearman 相关：

形状	实验室模型 ρ	实验室模型选中（实测）	波次 roofline ρ	波次 roofline 选中（实测）
4096^3	0.65	128×256×32（174.0）	0.26	256×256×32（75.5）
8192^3	0.36	128×256×32（211.4）	0.21	256×256×32（78.2）
256×4096^2	0.10	128×256×32（44.1）	0.72	64×128×32（151.7）

表 7: 两个解析代价模型的排序相关与它们选中配置的实测 TFLOP/s（`summary_gpu.json`）。

结论：每个解析模型只看见了一种效应

实验室模型（数据复用 + 缓存容量）在方阵上排序尚可（$\rho = 0.65$），因为方阵上主要矛盾是“小 tile 复用低”；但它对 M=256 几乎无信息（$\rho = 0.10$），选中的 128×256 只有 44.1 TFLOP/s——它不知道 256×4096 只能切出 $2 \times 16 = 32$ 个 C 块，170 个 SM 只用了 19%。波次 roofline 恰好相反：它抓住了 SM 占用（瘦长形状 $\rho = 0.72$），却因为不建模寄存器容量而在方阵上选中寄存器溢出的 256×256（75.5 TFLOP/s）。两个模型都不理解 warps/stages（流水线深度、寄存器分配），而这正是实测 5% 以内候选之间的区别。这就是 Ansor/TVM 用学习型代价模型 + 真机测量的原因：解析模型适合剪枝空间（去掉 OOR、明显过小的 tile），不适合做最终排序。

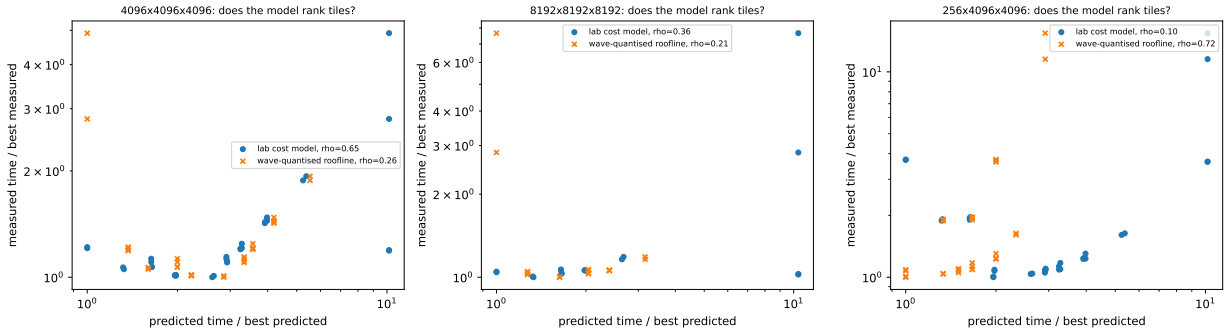


图 11: 预测时间（相对模型最优）对实测时间（相对实测最优），对数轴。理想模型应是单调曲线。右上角的离群点是实验室模型判为“工作集超出缓存”的 tile（预测 10×），实测确实是最差的一批之一。

4.4 Q3：量化——精度与速度

CPU 模型。 图 12：在 1024 通道中注入 8 个 50× 离群通道后，per-tensor W8A8 输出误差 10.2%，per-token 5.17%，SmoothQuant α=0.5 1.99%，LLM.int8 0.49%；GPTQ 在输入谱衰减 1.0 时测试误差 3.5%（RTN 25.3%），各向同性输入时反而更差（28.2% vs 26.4%）。这些合成实验预测了：(1) 离群是按通道出现的；(2) GPTQ 的收益取决于输入相关性。

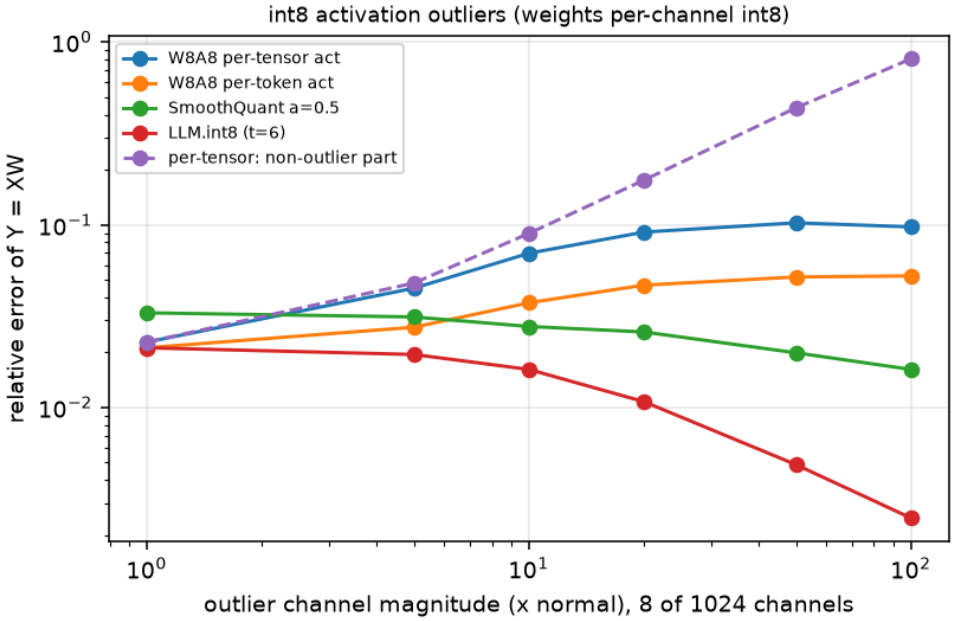


图 12: （CPU 模型）激活离群倍数对各 W8A8 方案输出误差的影响（`quant_outliers.csv`）。

GPU 实测：GPT-2 困惑度。 图 13 与表 8。

类别	方法	位宽	GPT-2 124M	GPT-2 355M	量化耗时（355M）
基线	fp16	16	29.96	21.72	–
int8 RTN	per-tensor	8	34.33	23.40	即时
	per-channel	8	29.98	21.72	即时
int4	RTN per-channel	4	38.40	25.54	即时
	GPTQ per-channel	4	32.59	23.19	36.5 s
	RTN g128	4	33.27	23.04	即时
	AWQ 式 g128	4	32.36	23.00	23.4 s
	GPTQ g128	4	**31.20**	**22.53**	36.6 s
int3 g128	RTN	3	65.92	32.45	即时
	AWQ 式	3	53.35	29.43	23.4 s
	GPTQ	3	**37.24**	**25.50**	36.5 s
W8A8（模拟）	激活 per-tensor	8	49.94	23.97	–
	激活 per-token	8	30.18	21.79	–
	SmoothQuant α=0.5 + per-tensor	8	407.25	26.53	1.1 s
	SmoothQuant α=0.8 + per-tensor	8	30.49	21.97	1.1 s
	SmoothQuant α=0.5 + per-token	8	30.15	21.91	1.1 s
	SmoothQuant α=0.8 + per-token	8	30.16	21.77	1.1 s
	LLM.int8（阈值 6）	8	**30.06**	**21.77**	–

表 8: wikitext-2 test 困惑度（286,720 个评估 token，`quant_ppl_gpt2*.jsonl`）。量化范围：所有 block 内线性层；嵌入与 lm_head 为 fp16。

结论：位宽越低，“用数据”的方法越值钱

int8 per-channel RTN 已经无损（29.98 vs 29.96；21.72 vs 21.72），per-tensor 则损失 4.4/1.7 ppl——与 CPU 实验的“重尾权重让 per-tensor 失效”一致。int4 g128 时 RTN 损失 3.3/1.3 ppl，GPTQ 把损失压到 1.24/0.81；到 int3，RTN 的 65.92 被 GPTQ 拉回 37.24（124M），AWQ 式缩放只到 53.35。GPTQ per-channel（32.59）甚至好于 RTN g128（33.27）：误差补偿抵得上 16 倍多的 scale。模型越大越耐量化：355M 的 int3 GPTQ 只损失 3.8 ppl（124M 为 7.3）。

结论：SmoothQuant 的迁移强度 α 是模型相关的超参数

GPT-2 124M 上 α=0.5 + per-tensor 激活的困惑度是 407.25，比不平滑的 49.94 还差 8×；α=0.8 则降到 30.49（损失仅 0.53）。355M 上同样 α=0.5 更差（26.53 vs 23.97），α=0.8 恢复到 21.97。$s_j = \max|X_j|^{\alpha} / \max|W_j|^{1-\alpha}$ 在 α 较小时对 $\max|W_j|$ 更敏感：若某输入通道的权重列很大而校准激活很小，$s_j \ll 1$，测试时该通道 x_j/s_j 被放大，反而成为新的 per-tensor 离群值。SmoothQuant 原文对不同模型使用 0.5–0.85 的 α，与此一致。per-token 激活量化本身已把 W8A8 的损失降到 0.2/0.07 ppl，而 LLM.int8 最接近 fp16（30.06/21.77），代价是每层平均 3.3/4.1 列、最多 50/60 列走 fp16 的不规则计算。

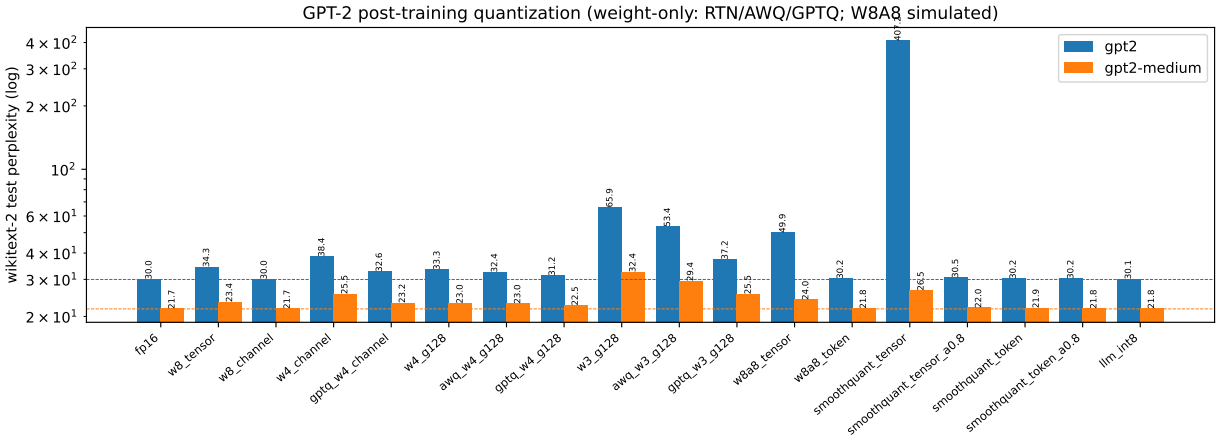


图 13: 各量化方法的困惑度（对数轴，虚线为 fp16）。

激活离群统计。 图 14。在 16 个校准窗口上统计每个线性层输入的逐通道 $\max|x|$。GPT-2 124M 最大的离群出现在第 2 个 block 的 `mlp.c_proj` 输入（GELU 之后）：62.8，而该层通道 $\max|x|$ 的中位数只有 1.60（39×）；355M 为第 3 个 block 的 `mlp.c_proj`，76.2 对中位数 1.78。`c_fc` 的输入（LN 之后）在多个 block 中是同一个通道 138 最大（124M 第 2 block 19.9、第 5 block 11.2），这就是文献中 GPT-2 残差流的固定离群维度经 LN 权重放大后的结果。超过阈值 6 的通道平均每层 4.0（124M）/ 5.9（355M）个，集中在靠后 block 的 `mlp.c_proj`（355M 第 21–23 block 每层 65–68 个）。这验证了 CPU 实验的前提：离群按通道出现、数量少（<2%）、幅度大。与 OPT-6.7B 之后出现的“系统性 >20× 离群”相比 GPT-2 的离群温和，所以朴素 W8A8 per-token 已经接近无损；离群对 per-tensor 激活才是致命的。

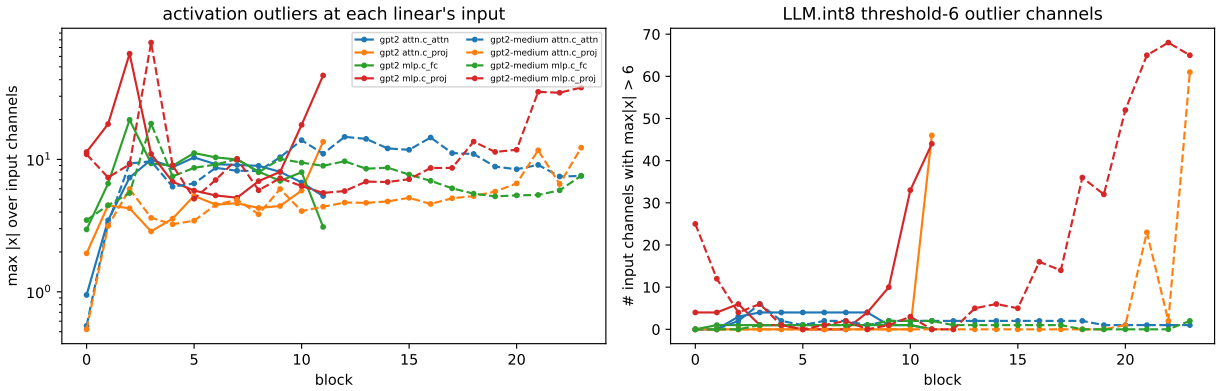


图 14: 左：每层输入的最大绝对激活（对数轴）；右：$\max|x| > 6$ 的输入通道数（LLM.int8 会把它们放到 fp16）。实线 124M，虚线 355M。

低精度 GEMM 吞吐。 图 15，`quant_speed.jsonl`。int8 结果与 fp64 参考逐元素完全相等（`exact=true`）；FP8 e4m3 相对 fp32 参考误差 1.66×10^{-3}。

$[M \times 4096] \times [4096 \times 16384]$	bf16 TFLOP/s	int8 TOP/s	fp8 TFLOP/s	int8/bf16	fp8/bf16
M=32（decode 式）	47.7	271.9	221.1	5.70×	4.64×
M=128	88.7	584.9	341.6	6.59×	3.85×
M=1024	177.4	599.9	383.0	3.38×	2.16×
M=8192（prefill 式）	226.9	679.4	456.2	2.99×	2.01×

表 9: 低精度 GEMM 吞吐（N=16384, K=4096）。

结论：在 GeForce Blackwell 上 int8 是 bf16 的 3 倍，不是 2 倍

大 M 时 int8 679.4 TOP/s、FP8 456.2 TFLOP/s、bf16 226.9 TFLOP/s：比例约 3:2:1。原因是 GeForce 卡的 bf16/fp16 在 fp32 累加时只有半速（规格 209.5 TFLOP/s），int8（int32 累加）是满速（规格 838 TOP/s，实测达到 81%），FP8 经 `_scaled_mm` 也是 fp32 累加（规格 419 TFLOP/s）。数据中心卡（H100）上 FP8 与 int8 都是 bf16 的 2×。小 M 时比例更大（M=32：int8 是 bf16 的 5.7×），因为 bf16 cuBLAS 在 M=32 选了一个低效的 kernel（47.7 TFLOP/s，90.0 µs），而权重读取 128 MB 在 1.79 TB/s 下的下限是 75 µs——bf16 的 decode 形状本来就接近带宽墙，int8 读一半字节（67 MB，15.8 µs 实测；部分命中 L2）。所以 W8A8 在 decode 上的收益主要来自少读字节，在 prefill 上来自更高的 *tensor core* 峰值。

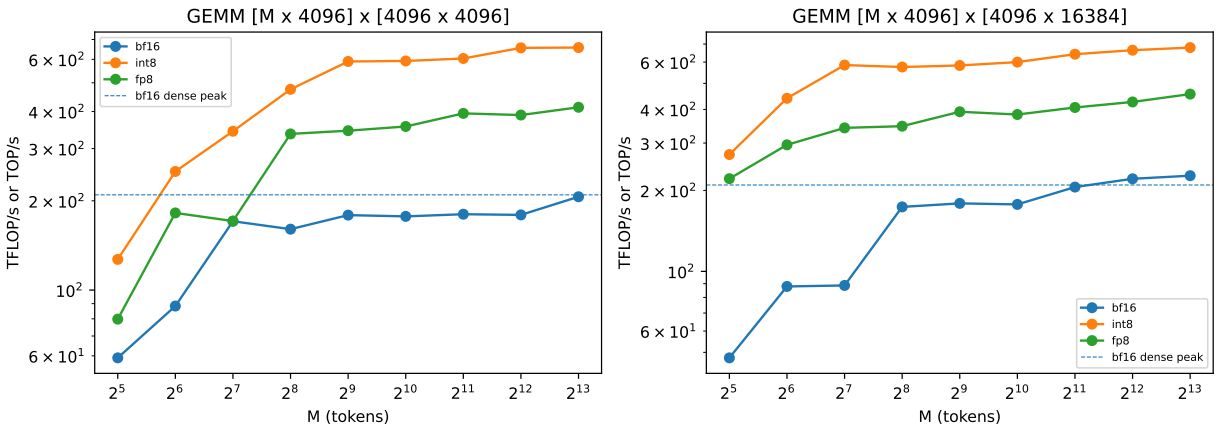


图 15: bf16 / int8 / FP8 GEMM 吞吐随 M 的变化（log-log）。注意 N=K=4096 时权重只有 32 MB（bf16），完全在 L2 中，所以小 M 的数字比“冷权重”乐观。

batch-1 weight-only：带宽说了算。 图 16。以 cold（轮换副本，工作集 > 384 MB）为准：

权重 $N \times K$	bf16 cuBLAS µs	W8A16 µs（加速）	W4A16 µs（加速）	先反量化再 matmul µs	bf16 TB/s
4096 × 4096	25.5	16.6（1.54×）	17.3（1.47×）	267.2	1.31
11008 × 4096	57.3	28.6（2.00×）	21.3（2.69×）	1096.8	1.57
16384 × 8192	163.0	82.5（1.98×）	62.2（2.62×）	3688.2	1.65
32768 × 8192	319.5	178.9（1.79×）	142.0（2.25×）	7454.3	1.68

表 10: batch 1 GEMV（cold），`quant_speed.jsonl`。W8/W4 与各自反量化参考的相对误差 2.1×10^{-4}。

结论：W8 接近 2×、W4 只到 2.3–2.7×，“先反量化”比融合 W4 kernel 慢 15–59×

bf16 GEMV 在大矩阵上达到 1.65–1.68 TB/s（规格峰值的 92–94%），已经是纯带宽受限；W8A16 读一半字节，加速 1.79–2.00×，接近理论 2×。W4A16 理论上限 3.76×（式 (5)），我们的 Triton kernel 只达到 2.25–2.69×：有效带宽 0.98–1.11 TB/s，瓶颈从 DRAM 转移到了解包（移位、掩码、int→fp 转换）与 x 的跨步读取——这正是 Marlin 等生产级 W4 kernel 花大量工程在 dequant 指令排布上的原因。“先在 PyTorch 里反量化成 fp16 再 cuBLAS”的做法比 bf16 直接算还慢 10–23×（多个 kernel、fp32 中间张量、反量化本身要写出完整 fp16 权重），即 weight-only 量化只有在融合进 *GEMV/GEMM kernel* 时才有速度收益。小矩阵（4096^2）三者都被 15–25 µs 的 launch 下限压平。

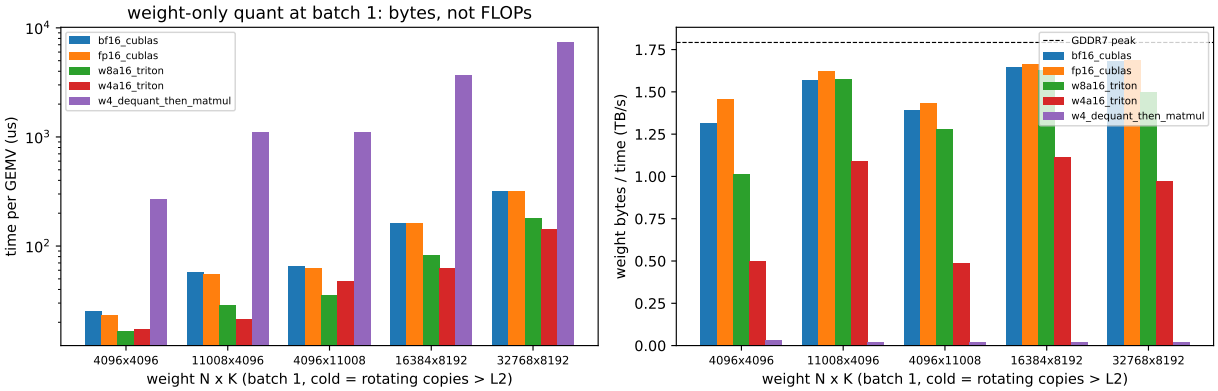


图 16: batch-1 GEMV 的时间（左，对数）与有效带宽（右，权重字节/时间），cold 模式。

4.5 Q4：剪枝——精度与稀疏 kernel 速度

CPU 模型。 图 17：1024^2 重尾矩阵上非结构化 50% 的输出误差 20.4%，2:4 为 29.0%，4:8 为 25.1%；小 MLP 上幅值剪枝 90% 不微调掉到 36.6%，微调 3 轮回到 63.6%（稠密 66.2%）。

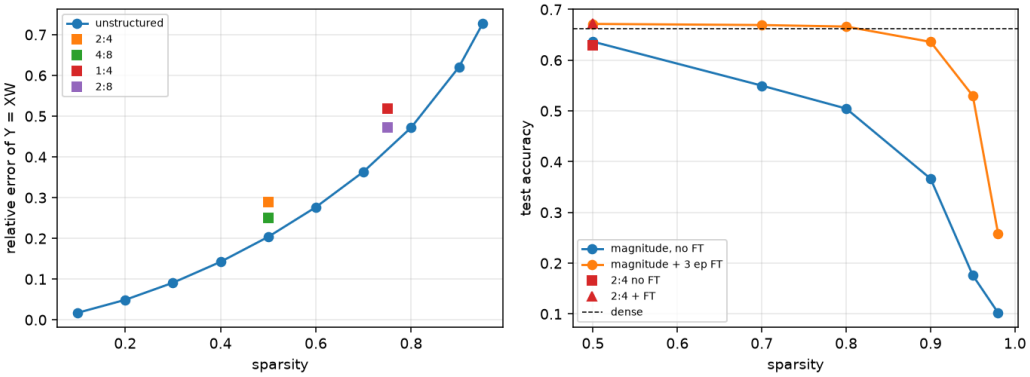


图 17: （CPU）左：剪枝方式对输出误差；右：小 MLP 的稀疏度—准确率，含/不含微调（`pruning_matrix.csv`、`pruning_mlp.csv`）。

GPU 实测。 图 18 与表 11。

方法（GPT-2 124M，一次性，无微调）	稀疏度	wikitext-2 ppl
稠密	0	29.96
幅值 30%	0.30	42.46
幅值 50%	0.50	538.35
幅值 70%	0.70	5698.52
幅值 2:4	0.50	735.95
幅值 4:8	0.50	346.51
Wanda 50%（按行）	0.50	**47.21**
Wanda 2:4	0.50	**116.38**

表 11: 一次性剪枝的困惑度（`prune_ppl_gpt2.jsonl`）。Wanda 的分数 $|W_{ij}| \cdot \|X_j\|_2$ 来自 64 个校准窗口。

结论：同样 50%，“看激活”比“看权重”好 11 倍

幅值剪枝 50% 把 ppl 从 29.96 推到 538.35，Wanda 只到 47.21；2:4 时 735.95 对 116.38。原因与图 14 相同：少数输入通道的激活幅度大几十倍，小权重乘以大激活并不小，只看 $|W|$ 会剪掉它们。N:M 约束越紧越差（4:8 346.51 好于 2:4 735.95），这和 CPU 实验的 25.1% vs 29.0% 同向。即使 Wanda 2:4 也损失 3.9× ppl——GPT-2 这样的小模型冗余少，生产中 2:4 需要配合再训练（NVIDIA 的 ASP 流程）。

W 形状（fp16）	M	稠密 µs	2:4 cuSPARSELt µs	加速
4096 × 4096	32	13.4	445.0	0.03×
4096 × 4096	2048	346.8	456.9	0.76×
4096 × 4096	8192	1244.8	806.6	1.54×
16384 × 4096	2048	1246.4	801.2	1.56×
16384 × 4096	8192	4936.4	3152.5	1.57×
4096 × 16384	8192	4947.6	3128.2	1.58×

表 12: 2:4 半结构化稀疏 GEMM（`prune_speed.jsonl`）。与稠密结果的相对误差 $\leq 3.6 \times 10^{-4}$。CUTLASS 后端在 sm_120 上报错 “*Supported only on GPUs with compute capability 8.x*”，只有 cuSPARSELt 可用。

结论：2:4 在大 GEMM 上兑现 1.54–1.58×，小 M 被约 445 µs 的固定开销吞掉

大形状上 cuSPARSELt 达到 340.8–351.5 有效 TFLOP/s（按稠密 FLOPs 计），即 1.54–1.58×，低于理论 2×（元数据读取、压缩格式的额外开销）。但每次调用有约 445 µs 的下限（M=32 时稠密仅 13.4 µs），使 $M \leq 2048$ 时 2:4 反而更慢——decode 场景完全不能用。非结构化 CSR（cuSPARSE）更糟：4096^2、M=512 时 50% 稀疏慢 18.5×（1815.1 vs 98.1 µs），要到 99% 稀疏才快 2.18×；batch 1 时任何稀疏度都不快于稠密（0.28–0.91×）。这就是 Deep Compression 时代的“剪枝省存储”与 GPU 上“剪枝省时间”的鸿沟：后者需要硬件认识的结构（2:4）与足够大的问题规模。

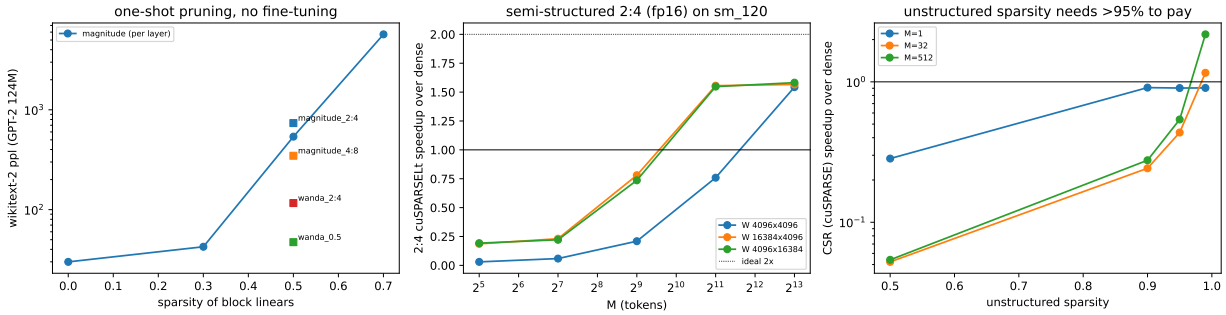


图 18: 左：GPT-2 一次性剪枝的困惑度（对数）；中：2:4 cuSPARSELt 相对稠密 cuBLAS 的加速随 M；右：CSR 相对稠密的加速随稀疏度（4096^2）。

4.6 Q5：LoRA 与 QLoRA-like 微调

CPU 模型。 单层 d=256、真实 ΔW 秩 8 的拟合：r=4 误差 0.434（Eckart–Young 下界 0.424），r=8 误差 0.044，可训参数 4096 vs 全量 65,536（6.25%）；初始输出与基座最大差 0，合并后最大差 3.3×10^{-15}（`lora_fit.csv`、`lora_params.csv`）。

GPU 实测（GPT-2 medium，wikitext-2，150 步，batch 8×512）。 图 19 and 20，`lora_ft.jsonl`。

配置	可训参数	占比	峰值显存 GiB	步前显存 GiB	步时 ms	最终 val loss
全量微调（AdamW, fp32 主权重）	354.8M	100%	16.07	1.98	128.4	2.891
LoRA r=4	1.57M	0.44%	12.10	1.34	103.8	2.885
LoRA r=8	3.15M	0.88%	12.13	1.35	104.3	2.875
LoRA r=16	6.29M	1.74%	12.17	1.36	104.8	**2.874**
LoRA r=32	12.58M	3.42%	12.27	1.38	104.8	2.877
LoRA r=64	25.17M	6.62%	12.45	1.43	107.0	2.901
QLoRA-like int8 基座 r=16	6.29M	1.74%	11.91	1.10	109.6	2.876
QLoRA-like int4 基座 r=16	6.29M	1.74%	11.77	0.95	115.5	2.914

表 13: 微调成本与效果（步时为第 10 步之后的中位数）。

结论：在这个规模上，LoRA 省的是优化器状态，峰值被激活主导

全量微调峰值 16.07 GiB，LoRA 约 12.1 GiB（−25%），差值 3.97 GiB 接近 AdamW 的两个 fp32 矩 + fp32 梯度（354.8M × 12 B = 4.26 GB = 3.97 GiB）——LoRA 省掉的正是这一块。剩下的约 10.7 GiB 是激活与 logits（8 × 512 token、vocab 50257，fp32 logits 一份 0.82 GB），与是否 LoRA 无关，所以 r 从 4 到 64 峰值只从 12.10 增到 12.45 GiB。int8/int4 冻结基座把“步前显存”从 1.36 降到 1.10/0.95 GiB（被量化的 block 内线性层共 302M 参数：0.60 GB bf16 → 约 0.30/0.16 GB），峰值只降 0.26/0.41 GiB：QLoRA 的价值在 7B–65B 模型上（基座本身 14–130 GB），在 355M 上几乎看不出。代价是步时：int8 基座 +4.6%、int4 +10.2%（前向与反向各反量化一次）。

结论：rank 不是越大越好；LoRA 在 150 步内追平甚至略好于全量

最终 val loss：全量 2.891，LoRA r=16 2.874、r=8 2.875、r=4 2.885，r=64 反而最差（2.901）——在固定 $\alpha/r = 2$ 与 lr 10^{-3} 下大 rank 的更新步更大、更易过拟合这 150 步的少量数据。int8 基座与 bf16 基座几乎相同（2.876 vs 2.874），int4 基座贵 0.040（2.914），对应表 8 中 W4 RTN 的精度损失——QLoRA 用 NF4 与双重量化来缩小这一差距，本实验用的是朴素 int4 g128。LoRA 每步比全量快 19%（103.8 vs 128.4 ms）：省掉 354.8M 参数的权重梯度计算与优化器更新，但激

活梯度的反向一点不少。

合并等价。 训练后的 r=8 模型，把每层换成 $W' = W + \frac{\alpha}{r}BA$ 的普通 `nn.Linear`，在 fp32 下与未合并模型比较 4 个验证窗口的 logits：最大绝对差 0.0045，而 logits 绝对值最大 340.6（相对 1.3×10^{-5}，fp32 累加顺序不同所致），argmax 一致率 100%。合并后推理零额外开销，这是 LoRA 相对 adapter 的核心卖点。

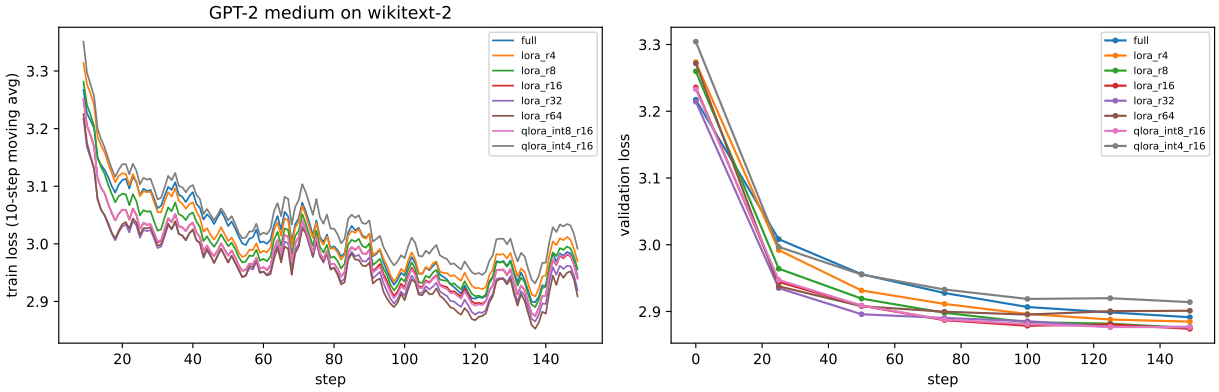


图 19: 训练损失（10 步滑动平均）与验证损失（每 25 步）。

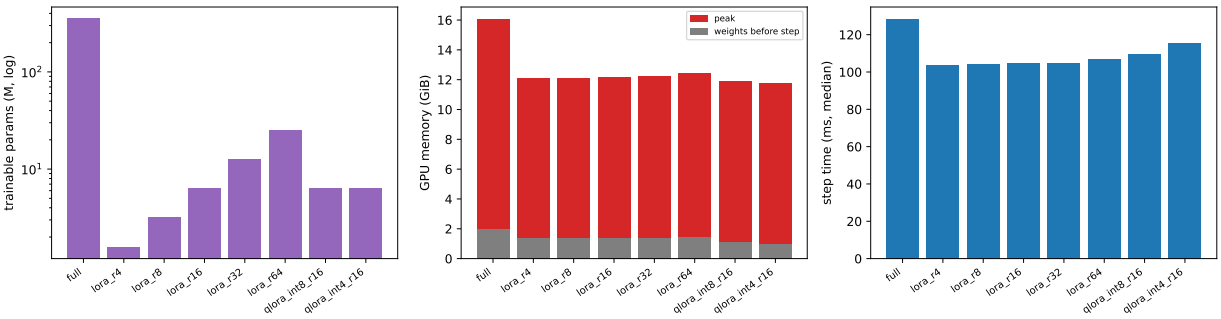


图 20: 可训参数（对数）、峰值/步前显存、步时。

4.7 Q3 扩展：换到 7B——Qwen2.5-7B 上的同一套量化（RTX 6000D）

GPT-2 的激活离群远弱于 6.7B 以上的模型，所以把同一份 `gpu/quant_llm.py` 原样跑在 Qwen2.5-7B 上（bf16 载入，wikitext-2 test 29.9 万 token，校准 128 × 1024 token，HF ctx 1024 协议）。原始结果在 `artifacts/raw/rtx6000d/quant_ppl_Qwen2.5-7B.jsonl`。

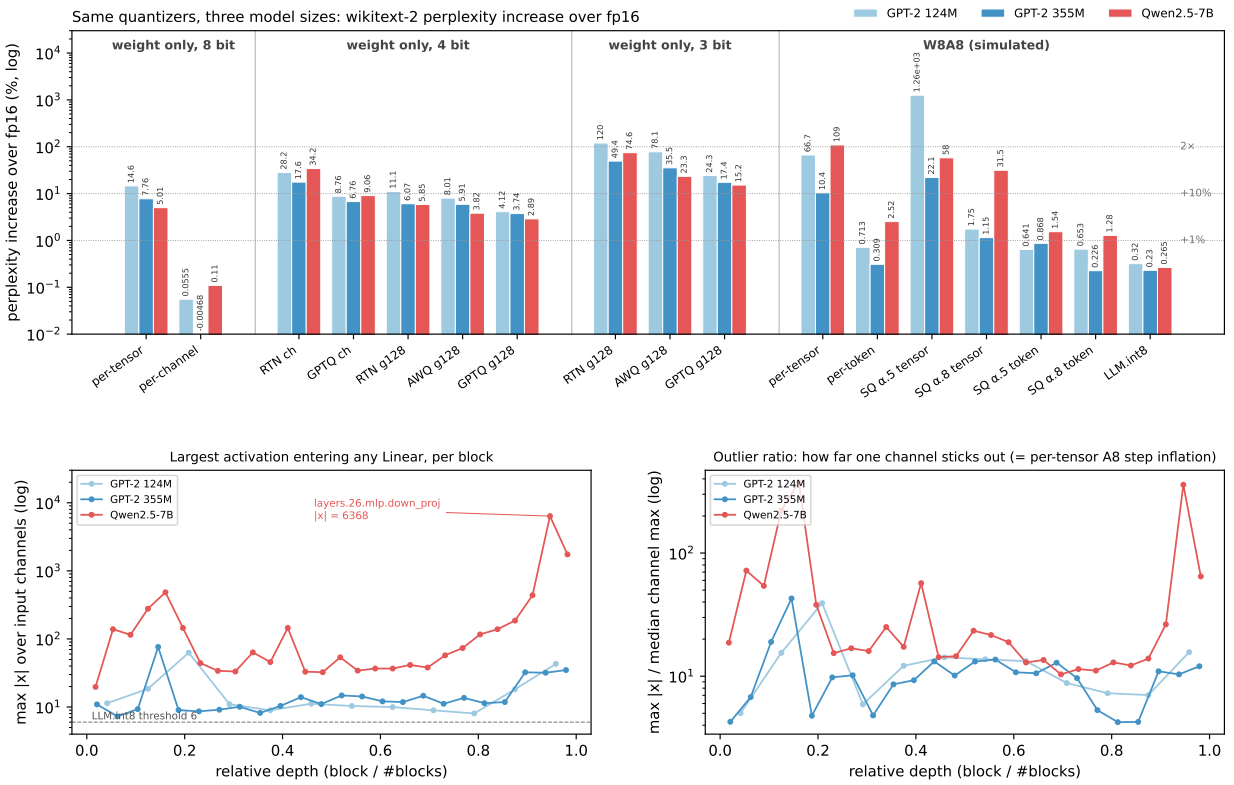


图 21: 三个规模上各方法相对 fp16 的困惑度增量，以及 Qwen2.5-7B 各层 Linear 输入的通道最大值（第 26 层 `down_proj` 有 |x| = 6368 的离群）。

表 14: 相对 fp16 的困惑度增量（Qwen2.5-7B fp16 为 7.639）

方法	GPT-2 124M	GPT-2 355M	Qwen2.5-7B
W8 逐张量 / 逐通道	+14.6% / +0.06%	+7.8% / −0.005%	+5.0% / +0.11%
W4 g128：RTN / AWQ / GPTQ	+11.1 / +8.0 / +4.1%	+6.1 / +5.9 / +3.7%	+5.9 / +3.8 / +2.9%
W3 g128：RTN / AWQ / GPTQ	+120 / +78 / +24%	+49 / +36 / +17%	+75 / +23 / +15%
W4 逐通道：RTN / GPTQ	+28 / +8.8%	+18 / +6.8%	+34 / +9.1%
W8A8 逐张量 / 逐 token	+67 / +0.71%	+10 / +0.31%	**+109** / +2.5%
SmoothQuant α=0.5 / 0.8（逐张量）	+1260 / +1.8%	+22 / +1.2%	+58 / +32%
SmoothQuant α=0.5 / 0.8（逐 token）	+0.64 / +0.65%	+0.87 / +0.23%	+1.5 / +1.3%
LLM.int8（阈值 6）	+0.32%	+0.23%	**+0.27%**

结论：权重量化随规模变容易，激活量化随规模变难

W4 g128 GPTQ 从 +4.1% 降到 +2.9%，W3 GPTQ 从 +24% 降到 +15%；W8A8 逐张量却从 GPT-2 355M 的 +10% 涨到 7B 的 +109%。Qwen 第 26 层 `mlp.down_proj` 的输入里有一个通道 |x| = 6368，是该层通道最大值中位数的 358 倍；逐张量的激活步长由它决定，其余通道只剩几个量化级。SmoothQuant 逐张量在 α=0.8 时仍 +32%：离群太大，平滑把难度挪给权重后权重也装不下；逐 token 激活加平滑才回到 +1.3%。LLM.int8 把 |x| > 6 的列留在 fp16，+0.27% 是所有 W8A8 方法中最好的，代价是平均 700 列、最多 18702 列走 fp16。

7B 上方法的排序被拉开：W3 g128 时 RTN +75%、AWQ +23%、GPTQ +15%。AWQ 按激活幅度缩放通道，在离群明显的模型上才有东西可缩放。GPTQ 第一次运行在 `down_proj`（18944 维输入）上报 Cholesky 非正定：1% 阻尼的 fp32 Hessian 被离群通道撑得条件数太大。`gptq_quantize`

现在用 float64 分解，失败时把阻尼放大 10 / 100 倍重试；三个 GPTQ 配置各 967 s。

不能推出的结论

困惑度协议（HF ctx 1024）与 edge-deploy llama-cpp-7b 的 llama-perplexity ctx 512 不同，两边数字不能直接比；W8A8 全部是精度模拟（fp16 GEMM），不含速度；只有一个 7B 模型，离群通道的位置与幅度是 Qwen2.5 的特性。

5 与理论模型的对照

5.1 融合：时间 = 字节 / 带宽

设一条逐元素/行归约链有 k 个 eager kernel，第 i 个读 r_i、写 w_i 个字节。eager 的总流量与融合后的下界为

$$B_{\text{eager}} = \sum_{i=1}^{k} (r_i + w_i), \qquad B_{\text{fused}} = \sum_{\text{图输入}} |x| + \sum_{\text{图输出}} |y|. \tag{1}$$

访存受限时 $T \approx B/\beta$，所以融合加速的上界就是 $B_{\text{eager}}/B_{\text{fused}}$。以 RMSNorm 为例：bf16 输入 x（S 字节）先上转 fp32（读 S 写 $2S$），平方（读 $2S$ 写 $2S$）、按行求均值（读 $2S$）、$x \cdot \text{rstd}$（读 $2S$ 写 $2S$）、乘权重（读 $2S$ 写 $2S$）、下转 bf16（读 $2S$ 写 S），合计约 $20S$ 量级，而融合只需 $2S$——dispatch 计数器给出 2685 MB / 268 MB = 10.0×，实测 9.86×（表 3）。β 取实测的 1.53 TB/s 时，M=16384 的理想 268 MB 需要 175 µs，与 Inductor 的 174.9 µs 吻合到 0.1%。

为什么 MLP 的融合收益小。 MLP 的 FLOPs 为 $2 \cdot T \cdot d \cdot 4d \cdot 2 = 16Td^2$，$T$=4096, d=1024 时 68.7 GFLOP，按 209.5 TFLOP/s 需 328 µs；可被融合省掉的是 LN 输出、$[T, 4d]$ 的 GELU 输入/输出与 $[T, d]$ 的残差前张量的写 + 读，实验室模型给出 293.6 MB → 125.8 MB，省 167.8 MB，按 1.79 TB/s 约 94 µs。所以全融合的上界是 (328 + 剩余访存) 相对 (328 + 94 + . . .)，即 $\leq 20\%$；实测 max-autotune 拿到 15%。RTX 5090 的脊点为

$$I^* = \frac{P}{\beta} = \frac{209.5 \times 10^{12}}{1.792 \times 10^{12}} \approx 117 \text{ FLOP/B}, \tag{2}$$

$[T, d] \times [d, 4d]$ 的 GEMM 强度约 $\frac{2 \cdot T \cdot d \cdot 4d}{2(Td + 4d^2 + 4Td)}$，$T$=4096, d=1024 时约 683 FLOP/B $\gg I^*$，计算受限；逐元素算子强度 < 1 FLOP/B，访存受限。

5.2 GEMM 分块：强度、共享内存与波次

一个 $B_M \times B_N$ 的 C 块沿 K 每步读 $(B_M + B_N)B_K$ 个元素、做 $2B_M B_N B_K$ 次运算，片上强度

$$I_{\text{tile}} = \frac{2B_M B_N B_K}{2(B_M + B_N)B_K} = \frac{B_M B_N}{B_M + B_N} \text{ FLOP/B (fp16)}. \tag{3}$$

64×64 只有 32 FLOP/B，低于 DRAM 脊点 117，却是 4096^3 的最优——因为 4096^3 的 A、B 各 32 MB，全在 96 MB L2 里，真正的“带宽”是 L2 带宽（数 TB/s 量级），脊点降到约 30–40 FLOP/B。这解释了为什么只按 DRAM 带宽建模的波次 roofline 偏爱大 tile 而排错。共享内存约束（Triton 软件流水线保留 $s - 1$ 份缓冲）：

$$(s - 1) \cdot (B_M + B_N) \cdot B_K \cdot 2 \text{ B} \leq 101376 \text{ B}, \tag{4}$$

例如 $32 \times 256 \times 64$、s=4 需要 $3 \cdot 288 \cdot 64 \cdot 2 = 110592$ B，正是报错信息中的数字。波次：$\lceil \#\text{tiles}/170 \rceil$；$M$=256、128×256 只有 32 个 tile，利用率 $32/170 = 19\%$。

5.3 weight-only 量化：batch 1 下时间 $\propto$ 权重字节

batch-1 GEMV 的 FLOPs 为 $2NK$，字节为 $NK \cdot b/8$（b 为权重位宽），强度 $= 16/b$ FLOP/B（bf16 为 1，int4 为 4），都远低于 117，因此

$$T_{\text{GEMV}} \approx \frac{NK\,b/8 + \text{scale 字节}}{\beta}, \qquad \text{加速}_{\text{W4 vs bf16}} \le \frac{16}{4 + 16/g} = 3.76\ (g = 128). \tag{5}$$

同理 decode 阶段的 W8A8 与 W4A16 之争：decode 是 GEMV，W4A16 读的字节少一半；prefill 是大 GEMM，W8A8 能用 int8 tensor core 提高峰值。

5.4 GPTQ 与 SmoothQuant 的公式

GPTQ 最小化层输出误差 $\|WX - \hat{W}X\|_F^2 = \text{tr}(\Delta W H \Delta W^\top)$，$H = XX^\top$。量化第 q 列后，用 OBS 的闭式更新把误差推给未量化列：

$$\delta_{:,F} = -\frac{w_{:,q} - \text{quant}(w_{:,q})}{[H^{-1}]_{qq}}\,[H^{-1}]_{q,F}. \tag{6}$$

用 H^{-1} 的上三角 Cholesky 因子可以按列顺序一次算完所有 $[H^{-1}]_{q,F}$，这是 GPTQ 相对 OBQ 的 $O(d^3)$ 加速来源。SmoothQuant 用 $XW^\top = (X\,\text{diag}(s)^{-1})(W\,\text{diag}(s))^\top$ 把激活的通道尺度挪给权重，$s_j = \max|X_j|^\alpha / \max|W_j|^{1-\alpha}$；该恒等式在测试 `test_smoothquant_identity_without_quant_noise` 中验证。

5.5 LoRA/QLoRA 的显存账

AdamW 全量微调每参数 16 B（fp32 权重 4 + 梯度 4 + 两个矩 8）。GPT-2 medium 354.8M 参数 $\Rightarrow$ 5.68 GB 静态状态。LoRA 冻结基座只需 bf16 权重 2 B/参数（0.71 GB），int8 1 B、int4 约 0.53 B（含 g128 scale）；可训参数 $r \cdot \sum_\ell (d_{\text{in}} + d_{\text{out}})$。但峰值显存还包括激活：batch 8×512、vocab 50257 时 fp32 logits 一份就是 $8 \cdot 512 \cdot 50257 \cdot 4 = 0.82$ GB，前向/softmax/反向要存多份。

6 面试专题

6.1 20 秒结论

> 在 RTX 5090 上，访存受限链的 `torch.compile` 加速正好等于 eager/融合的字节比（RMSNorm 10.0× 字节、9.86× 时间，融合后 1.53 TB/s 顶到带宽墙）；含 GEMM 的 MLP 只快 15%，且要用 max-autotune 付 11.7 倍编译时间。量化方面 int8 per-channel 无损，int3 时 GPTQ 把 GPT-2 的 ppl 从 65.9 拉回 37.2；batch-1 下 W8/W4 GEMV 快 1.8–2.0×/2.3–2.7×，只因为少读字节——前提是反量化融合进 kernel，否则慢 10–23×。

6.2 STAR 故事

S：要验证“图层融合能省多少时间”，CPU 上的模型预测 MLP 流量降 49% 但时间只降 16%。**T**：在真实 GPU 上测出 eager 与 Inductor 的字节、kernel 数、时间，判断模型是否可信、max-autotune 是否值得默认开启。**A**：写了 `TorchDispatchMode` 字节计数器给出 eager 字节；用 profiler 数 kernel；5 条访存受限链 × 4 个尺寸 × 4 种模式；把实验室图 IR 换成 5090 参数直接预测 MLP。**R**：访存受限链的实测加速与字节比相差 <7%；实验室模型预测 MLP 不融合 435.7 µs（实测 419.4），全融合 338.1 µs（实测 356.8）；同时发现 max-autotune 的 CUDA graph 输入拷贝让 128 MB 级的访存受限 kernel 慢 2.08×（364.0 vs 174.9 µs）——结论是 serving 的 decode 路径开 CUDA graph，prefill/训练的大张量路径用 `max-autotune-no-cudagraphs`。

6.3 追问

面试追问：torch.compile 为什么能让 RMSNorm 快 10 倍，却让 MLP 几乎不变？

RMSNorm 是一串逐元素/行归约算子，eager 每步都把 fp32 中间张量写回显存再读出（9 个 kernel，2685 MB），融合后只读 x、写 y（268 MB），访存受限时时间正比于字节，所以 9.86×。MLP 的 83% 以上时间在两个计算受限的 GEMM 里，cuBLAS 已经做了 bias epilogue，可融合的只有 GELU/残差的约 168 MB 往返，上界约 20%，实测 default 1.2%、max-autotune 15%。

常见错答：“compile 生成了更好的 kernel，所以都会变快”——它对 GEMM 默认仍调用 cuBLAS，收益只来自删掉中间张量。

面试追问：max-autotune 一定比 default 快吗？

不。它会开 CUDA graph（cudagraph trees），输入不是静态地址时每次 replay 前要拷贝输入：对 16384×4096 的 softmax，default 175.3 µs，max-autotune 363.4 µs，no-cudagraphs 175.2 µs。它还要 benchmark GEMM 模板，MLP 编译 6.9 s、block 12.9 s，并且很多为 H100 设计的模板（128–144 KB 共享内存）在 sm_120 的 99 KB 上直接失败。

常见错答：“max-autotune 只是多试几个配置，最差也和 default 一样”。

面试追问：动态 shape 下 torch.compile 怎么避免反复重编译？

默认 automatic dynamic：第一次按静态 shape 编译，看到同一维度变化后编一个符号形状图（我们 10 个长度共 2 个图、0.99 s）；`mark_dynamic` 一开始就编符号图（1 个图、0.54 s）；`dynamic=False` 每个新长度都编（8 个图到 `cache_size_limit`，4.46 s）。对 GEMM 主导的 MLP，符号形状图稳态无损失（4096 token 时 420.2 vs 419.8 µs）。

常见错答：“dynamic shape 一定慢，所以应该全静态”——代价在编译与 guard，不在 kernel。

面试追问：Triton matmul 的 tile 怎么选？代价模型靠得住吗？

看三件事：片上强度 $B_M B_N/(B_M+B_N)$（够用即可，A/B 在 L2 里时脊点低）、共享内存 $(s-1)(B_M+B_N)B_K \cdot 2 \le 99$ KB、C 块数相对 170 个 SM 的波次与占用；寄存器（256×256 fp32 累加器 256 KB）也是硬约束。实测最优在 4096^3 是 64×64、8192^3 是 128×128、M=256 是 128×64；复用型解析模型 $\rho = 0.65/0.36/0.10$，波次模型 $\rho = 0.26/0.21/0.72$，各只看见一种效应，所以 Ansor 用学习型模型 + 实测。

常见错答：“tile 越大复用越高越快”——256×256 在三个形状上都最差（75/78/14 TFLOP/s）。

面试追问：GPTQ 比 RTN 好在哪？什么时候没用？

GPTQ 最小化 $\mathrm{tr}(\Delta W H \Delta W^\top)$，每量化一列就用 H^{-1} 把误差补偿到未量化列上。int3 g128 时 GPT-2 124M 的 ppl RTN 65.92 → GPTQ 37.24。它依赖输入相关性：CPU 实验中各向同性输入（$H \approx cI$）下 GPTQ 测试误差 28.2% 反而比 RTN 26.4% 差，补偿只拟合了校准噪声。

常见错答：“GPTQ 让量化后的权重更接近原权重”——它的权重误差更大（CPU 实验 0.28–0.39 vs 0.26），它优化的是输出。

面试追问：W4A16 和 W8A8 选哪个？

decode（batch 1–小 batch）是 GEMV，强度 $16/b$ FLOP/B 远低于脊点 117，时间 $\propto$ 权重字节：W4A16 在 16384×8192 上 62.2 µs vs bf16 163.0 µs（2.62×）。prefill 是大 GEMM，计算受限，W8A8 能用 int8 tensor core：M=8192 时 int8 679.4 TOP/s vs bf16 226.9（3.0×，GeForce 的 bf16 fp32 累加是半速）。

常见错答：“int4 比 int8 少一半位，所以永远快一倍”——W4 的解包开销让我们的 kernel 只到 0.98–1.11 TB/s，且 prefill 时 W4A16 仍要用 bf16 tensor core 计算。

面试追问：SmoothQuant 的 α 怎么选？

α 决定把多少激活难度迁移到权重。GPT-2 124M 上 α=0.5 + per-tensor 激活得到 ppl 407.25（比不平滑的 49.94 还差），α=0.8 得到 30.49。它是模型相关的超参数，应在校准集上扫；若激活能做 per-token，W8A8 本身已近无损（30.18）。

常见错答：“论文用 0.5，所以 0.5 是通用最优”。

面试追问：为什么非结构化剪枝在 GPU 上难以变快，2:4 呢？

非结构化稀疏打乱了 tensor core 需要的稠密 tile，只能走 CSR 的间接寻址：4096^2、M=512、50% 稀疏时 cuSPARSE 慢 18.5×，99% 才快 2.18×。2:4 每 4 个元素留 2 个、用 2 bit 元数据记位置，稀疏 tensor core 直接跳过一半 MAC：大 GEMM 上实测 1.54–1.58×；但 cuSPARSELt 在 sm_120 上每次调用约 445 µs 的开销，使 $M \le 2048$ 时更慢。精度上，同为 50%，幅值 2:4 让 GPT-2 ppl 变 735.95，Wanda 2:4 为 116.38。

常见错答：“剪掉 50% 权重就省 50% 时间”——要硬件支持的结构、足够大的 M，且精度损失要靠激活感知或再训练补回。

面试追问：LoRA 省的显存到底在哪？QLoRA 又省在哪？

全量 AdamW 每参数 16 B：GPT-2 medium 的 fp32 梯度 + 两个矩约 3.97 GiB，正好是全量（16.07 GiB）与 LoRA（12.10–12.45 GiB）峰值之差；剩下的 10.7 GiB 是激活，LoRA 不省。QLoRA 再把冻结基座从 2 B/参数压到约 0.5 B/参数，对 355M 只省 0.41 GiB，对 65B 模型则是 130 GB → 约 35 GB，决定能否上单卡。注意实现细节：反量化出的权重若交给 `F.linear`，autograd 会把它存进图里，显存一点不省；要用自定义 `autograd.Function` 只存量化张量、反向时重算。

常见错答：“LoRA 可训参数只有 0.9%，所以显存也只要 1%”——激活与基座权重都还在。

面试追问：如何测一个“访存受限”kernel 的真实带宽而不被 L2 骗？

工作集要大于 L2（RTX 5090 为 96 MB），或者轮换多份副本。我们的 LayerNorm 4096×4096（67 MB）eager 显示 3.32 TB/s，高于 1.79 TB/s 的 GDDR7 峰值，就是 L2 命中；GEMV 的 cold 模式在 11008×4096 W8 上把“带宽”从 2.67 TB/s（hot）拉回 1.57 TB/s。

常见错答：“超过峰值说明计时有 bug”——是缓存层次，不是 bug。

不能推出的结论

- W8A8、SmoothQuant、LLM.int8 是“量化-反量化”的精度模拟，用 fp16 GEMM 计算，没有测这些方案的真实 int8 端到端速度；速度只在单个 GEMM/GEMV 上测。
- 困惑度只在 GPT-2 124M/355M 上测；它们的激活离群远弱于 6.7B 以上模型，LLM.int8/SmoothQuant 的必要性在大模型上会更突出。不能把“per-token W8A8 近无损”外推到 LLaMA/OPT 大模型。
- AWQ 是简化版（只在 $\overline{|x|}^{a}$ 上网格搜索，代理误差选 a，无 clip 搜索），GPTQ 无 act-order；两者的排名不代表原论文实现的排名。
- 低精度 GEMM 的 N=K=4096 权重（32 MB）常驻 L2，小 M 的吞吐偏乐观；GEMV 区分了 hot/cold，GEMM 没有。
- 融合实验的“理想字节”是下界；Inductor 的真实 DRAM 流量未用 Nsight Compute 测量，带

宽数字是“理想字节 / 时间”。小尺寸数字被 Python/guard 开销（11–27 µs）主导。

- Triton matmul 网格只覆盖 4 个超参维度，GROUP_M 固定 8，未试 persistent/TMA/warp-specialized kernel；“超过 cuBLAS”只在 fp16 方阵上成立，瘦长形状 cuBLAS 仍快 6%。
- 编译时间受服务器只给 2 个 CPU 核的影响，绝对值偏大；但模式之间的比例可信。
- 剪枝只做了一次性剪枝、未再训练；2:4 的 445 µs 调用下限可能是 PyTorch 2.8 的 cuSPARSELt 路径在 sm_120 上的实现问题，不应当作硬件特性。
- LoRA 只训练 150 步、单次种子；loss 差 0.01 量级在噪声范围内，“LoRA 略好于全量”不能推广，结论只有“在该预算下不比全量差”。
- RTX 5090 的峰值取规格值；实测 GEMM（221.5 TFLOP/s fp16、679.4 TOP/s int8）说明卡实际频率高于标称 boost，百分比应以实测上界理解。

7 结论

1. **融合的收益可以先算再测**：访存受限时加速 $\approx B_{\text{eager}}/B_{\text{fused}}$，实测吻合到 7% 以内；计算受限时上界由“可删的中间张量字节 / 总时间”决定，MLP 仅 15–20%。实验室的 CPU 图 IR 在换上 RTX 5090 参数后对 MLP 的预测误差为 $-3.7\%/+5.5\%$。
2. **编译器的工程代价是真实的**：max-autotune 编译时间是 default 的 11.7–22×，CUDA graph 对大张量是负优化，动态 shape 应 `mark_dynamic`（1 个图 0.54 s vs 静态 8 个图 4.46 s）。
3. **调度搜索需要实测**：解析代价模型的排序相关只有 0.10–0.72，且不同模型在不同形状上失效；它们适合剪枝搜索空间（OOR、过小 tile），最终选择要靠 benchmark（autotune）或学习型模型（Ansor）。
4. **量化：粒度 + 数据**：per-channel 解决权重重尾，int4 以下必须用数据（GPTQ 的 Hessian 补偿、AWQ 的激活缩放）；激活离群按通道出现，per-token 或 LLM.int8 分解最稳，SmoothQuant 需要调 α。
5. **速度只来自字节或峰值**：batch 1 下 weight-only 量化按字节比加速，但必须融合反量化；prefill 下 int8 tensor core 给出 3× 峰值；2:4 在大 GEMM 上 1.5×，非结构化稀疏在 GPU 上要 >95% 才划算。
6. **LoRA 省的是优化器状态**：355M 模型上峰值显存 -25%、步时 -19%，激活不变；QLoRA 的基座压缩要到数十 B 参数才是决定性的。

参考文献

[1] T. Chen et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. OSDI 2018.

[2] L. Zheng et al. Ansor: Generating High-Performance Tensor Programs for Deep Learning. OSDI 2020.

[3] Z. Jia et al. TASO: Optimizing Deep Learning Computation with Automatic Generation of Graph Substitutions. SOSP 2019.

[4] XLA: Optimizing Compiler for Machine Learning. Google, 2017; and A. Sabne, XLA: Compiling Machine Learning for Peak Performance, 2020.

[5] J. Ansel et al. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. ASPLOS 2024.

[6] P. Tillet, H. T. Kung, D. Cox. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. MAPL 2019.

[7] J. Ragan-Kelley et al. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. PLDI 2013.

[8] T. Dettmers et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. NeurIPS 2022.

[9] G. Xiao et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. ICML 2023.

[10] E. Frantar et al. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. ICLR 2023.

[11] E. Frantar, D. Alistarh. Optimal Brain Compression: A Framework for Accurate Post-Training Quantization and Pruning. NeurIPS 2022.

[12] J. Lin et al. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. MLSys 2024.

[13] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.

[14] T. Dettmers et al. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023.

[15] S. Han, H. Mao, W. J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. ICLR 2016.

[16] A. Mishra et al. Accelerating Sparse Deep Neural Networks. arXiv:2104.08378, 2021.

[17] M. Sun et al. A Simple and Effective Pruning Approach for Large Language Models (Wanda). ICLR 2024.

[18] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. CACM 2009.

[19] A. Radford et al. Language Models are Unsupervised Multitask Learners. OpenAI, 2019.

[20] S. Merity et al. Pointer Sentinel Mixture Models. ICLR 2017.